

รายงานแลป OLTP OLAP และ Pivot

ชื่อ: นายชลาส ศรเกษม รหัส: 67160211 กลุ่ม: 2

1 OLTP

โค้ดที่แก้ไข (oltp_demo.py):

```
cur = con.execute(
    "UPDATE orders SET status='PAID' WHERE order_id='O1004' AND status='PENDING'"
)
print('rowcount:', cur.rowcount)
con.commit()
```

ผลลัพธ์สองรอบ:

รอบ	Before	rowcount	After
1	('O1004','PENDING')	1	('O1004','PAID')
2	('O1004','PAID')	0	('O1004','PAID')

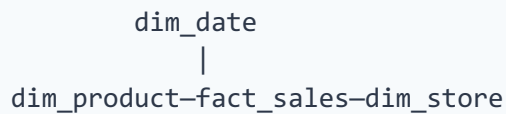
คำอธิบาย:

- รอบแรกแถวเปลี่ยนได้เพราะสถานะปัจจุบันตรงเงื่อนไข status='PENDING' ในตอนนั้น
- รอบสอง status ถูกเปลี่ยนเป็น PAID ไปแล้ว จึงไม่มีแถวใดตรงเงื่อนไขทั้งสองอีก ผลคือ rowcount = 0 โดยไม่เกิด error — นี่คือคุณสมบัติ idempotent ของ UPDATE ที่มีเงื่อนไขคลุม
- ต้องตรวจสอบ status ควบคู่กับ order_id เพื่อป้องกันไม่ให้สั่งเปลี่ยนสถานะซ้ำ (เช่น เปลี่ยนออเดอร์ที่ถูกยกเลิกแล้วกลับไปเป็น PAID โดยไม่ตั้งใจ) — เงื่อนไขทำหน้าที่เป็น "guard" ของ transaction
- กิจกรรมนี้ยังถือเป็น OLTP แม้จะมี SELECT เพราะ SELECT ที่ใช้เป็นเพียงการอ่านสถานะปัจจุบันของ "หนึ่งเรคคอร์ด" เพื่อสนับสนุนการทำธุรกรรม (transaction) ไม่ใช้การอ่านเพื่อวิเคราะห์/สรุปยอดจำนวนมากแบบ OLAP ลักษณะงานคือ อ่าน-เขียนทีละแถว ความถี่สูง สอดคล้องกับงานประมวลผลรายการ (transaction processing)
- การแก้ไข oltp.db ไม่ทำให้ warehouse.db เปลี่ยนทันที เพราะเป็นคนละไฟล์ฐานข้อมูล ไม่ได้เชื่อมกันอัตโนมัติ (ตามที่ README ระบุว่า oltp.db "ไม่ได้ทำ ETL อัตโนมัติ")

- ระบบจริงควรใช้กระบวนการ ETL/ELT (Extract-Transform-Load) เป็นรอบ (batch) หรือ CDC (Change Data Capture) แบบใกล้เคียงเวลา เพื่อดึงข้อมูลสถานะจากระบบ OLTP ไปปรับปรุงคลังข้อมูล (data warehouse) อย่างมีการควบคุมและตรวจสอบคุณภาพข้อมูล

2 Grain และ Star Schema

Star Schema:



- PK: fact_sales (order_id, line_no) — composite key
- FK: date_key → dim_date, product_key → dim_product, store_key → dim_store
- Relationship: 1:N จาก dim_date, dim_product, dim_store ไปยัง fact_sales (1 วัน/สินค้า/สาขา ผูกกับหลายแถวใน fact_sales)

Grain: 1 แถวใน fact_sales คือ 1 บรรทัดสินค้า 1 รายการ ภายใน 1 ออเดอร์ (order line) เช่น O1001 มี 2 แถว เพราะซื้อ Tea และ Cookie ในออเดอร์เดียวกัน — ไม่ใช่ "1 ออเดอร์ = 1 แถว"

Dimensions (3): dim_date, dim_product, dim_store

Measures (2): quantity (จำนวนชิ้น), amount (ยอดขาย = quantity × unit_price)

Hierarchy เวลา: year → month → full_date

Hierarchy สถานที่: region → province → store_name

ทำไม unit_price ไม่ควรบวกเป็นยอดขาย: unit_price เป็น "ราคาต่อหน่วย" (a rate) ไม่ใช่ยอดรวม

การนำราคาต่อชิ้นจากหลายแถวมาบวกกันจะได้ตัวเลขที่ไม่มีความหมายทางธุรกิจ (ผลรวมราคาที่ไม่ได้คูณกับจำนวนชิ้น) ต้องคูณกับ quantity ก่อนจึงจะรวมกันได้ตามสูตร amount = quantity × unit_price ซึ่ง amount คือยอดขายที่บวกได้ (additive) ในทุกมิติ

ผล q01.sql (ตรวจกับ warehouse.csv):

line_count	order_count	units	revenue
8	6	23	1390

ตรงกับข้อมูลในสไลด์ที่ระบุว่า มี 8 รายการ 6 ออเดอร์ รวม 1,390 บาท ✓

3 OLAP

ไฟล์	ผลลัพธ์	Operation
q02	2026-08: 490, 2026-09: 900	Roll-up (รายเดือน)
q03	Bangkok/Chonburi × เดือน (4 แถว)	Roll-up หลายมิติ (เดือน+จังหวัด)
q04	09-09: 360, 09-10: 300, 09-11: 240	Drill-down (เดือน→วัน)
q05	Bangkok 540, Chonburi 360	Slice (เฉพาะกันยายน)
q06	Bangkok 300, Chonburi 200	Dice (กันยายน + Drink + 2 จังหวัด)
q07	Bangkok 540 เท่านั้น	Slice + HAVING filter
q12	O1005 (300), O1006 (240)	Drill-through

WHERE vs HAVING: WHERE กรองแถวต่อก่อนจัดกลุ่ม (เช่น กรองเฉพาะเดือนกันยายน) ส่วน HAVING กรองผลลัพธ์หลัง GROUP BY แล้ว (เช่น เอาเฉพาะจังหวัดที่ยอดรวม > 400) — q07 จึงต้องกรองเดือนด้วย WHERE (เงื่อนไขระดับแถว) และกรองยอด >400 ด้วย HAVING (เงื่อนไขระดับกลุ่ม)

คำตอบคำถามระหว่างทำ:

- ก. q03 เพิ่ม "จังหวัด" ทำให้เปรียบเทียบยอดขายพื้นที่ในแต่ละเดือนได้ ส่วน q04 เพิ่มรายละเอียด "วัน" ภายในเดือนกันยายน ทำให้เห็นแนวโน้มรายวัน — ก. คือมิติสถานที่ (เปรียบเทียบเชิงพื้นที่), ข. คือมิติเวลา (เปรียบเทียบเชิงแนวโน้ม)
- ข. กรองเดือนด้วย WHERE (เงื่อนไขแถว) และกรองยอดรวมด้วย HAVING (เงื่อนไขหลัง SUM)
- ค. ผลรวมของ q12 (300+240=540) ต้องเท่ากับ Bangkok × 2026-09 ใน q03 ซึ่งเท่ากับ 540 ✓ ตรงกัน

จุดตรวจผ่าน: ยอดรายเดือน (q02) 490+900=1390 = รวมทั้งชุด (q01) ✓ | ยอดรายวันกันยายน (q04) 360+300+240=900 = ยอด Slice กันยายน (q05) 540+360=900 ✓

4 Pivot

q08.sql (province × เดือน แบบ SQL):

province	aug	sep	total
Bangkok	310	540	850
Chonburi	180	360	540

P1 (pivot_province_month.csv):

province	2026-08	2026-09	Total
Bangkok	310	540	850
Chonburi	180	360	540
Total	490	900	1390

P2 (pivot_september.csv, เฉพาะกันยายน):

category	Bangkok	Chonburi
Drink	300	200
Snack	240	160

Assert: `p1.loc['Total','Total'] == df['amount'].sum()` → `1390 == 1390` ✓ ผ่าน (เลือกเฉพาะเซลล์ Total×Total ไม่ใช่บวกทุกเซลล์ที่มีคำว่า Total)

ทดลอง Excel/Drink filter: ยอดรวมหลังกรอง Drink = 750 บาท (Tea 4 รายการ: 100+150+200+300)

แกน Rows = province, Columns = month, Values = amount (Sum)

province	2026-08	2026-09
Bangkok	150	300
Chonburi	100	200

ทดลอง Error (ลบ aggfunc): เมื่อไม่ระบุ aggfunc pandas ใช้ค่า default คือ mean ไม่ใช่ sum ดังนั้น Bangkok เดือนกันยายนที่มี 2 แถว (O1005=300, O1006=240) จะได้ค่าเฉลี่ย = $(300+240)/2 = 270$ ซึ่งคือ "ค่าเฉลี่ยต่อรายการ" ไม่ใช่ยอดขายรวม แก้กลับเป็น `aggfunc='sum'` จึงได้ 540 ที่ถูกต้อง

ข้อแตกต่าง sum กับ mean: sum คือผลรวมของทุกแถวในกลุ่ม (ใช้หา "ยอดขายรวม") ส่วน mean คือค่าเฉลี่ยต่อแถว (ใช้ดู "ขนาดเฉลี่ยต่อรายการ") ทั้งสองมีความหมายต่างกันและนำไปใช้แทนกันไม่ได้

5 ตรวจสอบความถูกต้อง

q09 (UNION ALL):

month	revenue
2026-08	490
2026-09	900
ALL	1390

หากนำทุกแถวที่ได้ (490+900+1390) มาบวกกันอีกครั้ง จะเกิดการนับยอดซ้ำ (double counting) เพราะแถว ALL เป็นผลรวมของอีกสองแถวอยู่แล้ว การบวกซ้ำจะทำให้ยอดสูงเกินจริงเป็น 2 เท่าของยอดจริง — ต้องเลือกใช้อย่างใดอย่างหนึ่ง (รายเดือน หรือ ALL) ไม่ใช่รวมทั้งหมด

q10:

revenue	orders	aov	avg_line
1390	6	231.67	173.75

หน่วยของ AOV: บาท/ออเดอร์ (Average Order Value) คำนวณจาก $\text{SUM}(\text{amount}) / \text{COUNT}(\text{DISTINCT order_id}) = 1390 / 6 = 231.67$ บาทต่อออเดอร์ ส่วน $\text{AVG}(\text{amount}) = 1390 / 8 = 173.75$ คือค่าเฉลี่ย "ต่อบรรทัดสินค้า" (per line) ไม่ใช่ต่อออเดอร์ — ตัวหารต่างกัน ($\text{COUNT}(\text{DISTINCT order_id})=6$ กับจำนวนแถว=8) จึงให้ค่าต่างกัน AOV จึงสูงกว่า avg_line เพราะบางออเดอร์มีหลายบรรทัด

q11 (ก่อน/หลัง JOIN):

rows_before_join	revenue_before_join	rows_after_join	revenue_after_join
8	1390	8	1390

PRAGMA foreign_key_check ค้นค่าว่าง (ไม่มี FK ที่ผิดพลาด) ยอดก่อน/หลัง JOIN เท่ากันเพราะข้อมูลสมบูรณ์ ไม่มี key ข้ำในมิติและไม่มี FK กำพรัว

ผลกระทบสมมติ:

- ถ้ามี key ข้ำในมิติ (เช่น store_key=1 ปรากฏ 2 แถวใน dim_store) การ JOIN จะสร้างแถวซ้ำ (fan-out) ทำให้จำนวนแถวและยอดขายหลัง JOIN สูงเกินจริง
- ถ้ามี FK ที่ไม่มีข้อมูลอ้างอิง (orphan FK) แถวนั้นจะหายไปจาก INNER JOIN ทำให้จำนวนแถวและยอดขายหลัง JOIN ต่ำกว่าความจริง

ตัวอย่าง Measure แต่ละประเภท:

ประเภท	ตัวอย่าง	วิธี Aggregate ที่เหมาะสม
Additive	amount (ยอดขาย)	SUM ได้ทุกมิติ (เวลา สินค้า สถานที่)
Semi-additive	สต็อกสิ้นวัน (ending stock)	SUM ข้ามสินค้า/สาขาได้ แต่ ข้ามเวลาไม่ได้ ต้องใช้ค่า ณ วันล่าสุด หรือ AVG แทน
Non-additive	AOV	ห้ามบวกหรือหาค่าเฉลี่ยแบบตรงไปตรงมาข้ามเดือน ต้องคำนวณใหม่จาก SUM(amount รวม)/COUNT(DISTINCT order_id รวม) ของช่วงเวลานั้น ๆ

6 สรุป

ข้อค้นพบ:

- เดือนกันยายนมียอดขายสูงกว่าเดือนสิงหาคม (900 บาท เทียบกับ 490 บาท) และ Bangkok มียอดขายรวมสูงกว่า Chonburi ในทั้งสองเดือน (850 เทียบกับ 540)
- AOV (231.67 บาท/ออเดอร์) สูงกว่า AVG(amount) แบบรายบรรทัด (173.75 บาท/บรรทัด) อย่างชัดเจน สะท้อนว่าออเดอร์ส่วนหนึ่งมีมากกว่า 1 รายการสินค้า การเลือกตัวหารที่ถูกต้อง (COUNT(DISTINCT order_id) แทนจำนวนแถว) จึงสำคัญมากต่อความหมายของตัวชี้วัด

ข้อจำกัด: ชุดข้อมูล warehouse.db มีเพียง 6 ออเดอร์ 8 บรรทัด ครอบคลุมแค่ 2 จังหวัดและ 2 เดือน ขนาดตัวอย่างเล็กเกินกว่าจะสรุปแนวโน้มเชิงธุรกิจที่กว้างขึ้นได้ และเป็นข้อมูลจำลองทั้งหมด (ไม่ใช่ยอดขายจริง) จึงยืนยันแนวโน้มตลาดจริงไม่ได้

การใช้ AI: ใช้ Claude ช่วยตรวจสอบตรรกะของ SQL (WHERE/HAVING/GROUP BY) และช่วยเขียนโครงโค้ด Python สำหรับ Pivot Table โดย Prompt สำคัญที่ใช้คือ "ให้ยึดข้อมูล ตาราง ชื่อคอลัมน์ตามไฟล์ที่แนบมาเท่านั้น ห้ามสร้างข้อมูลสมมติขึ้นเอง" จุดที่ตรวจสอบและแก้ไขด้วยตนเองคือ การรันได้จริงเทียบกับ warehouse.csv ทุกข้อ เพื่อยืนยันว่าตัวเลขที่ AI สร้างให้ตรงกับข้อมูลจริง ไม่ใช่ค่าที่เดาขึ้นมา